

Making 3D Games with AI Agents



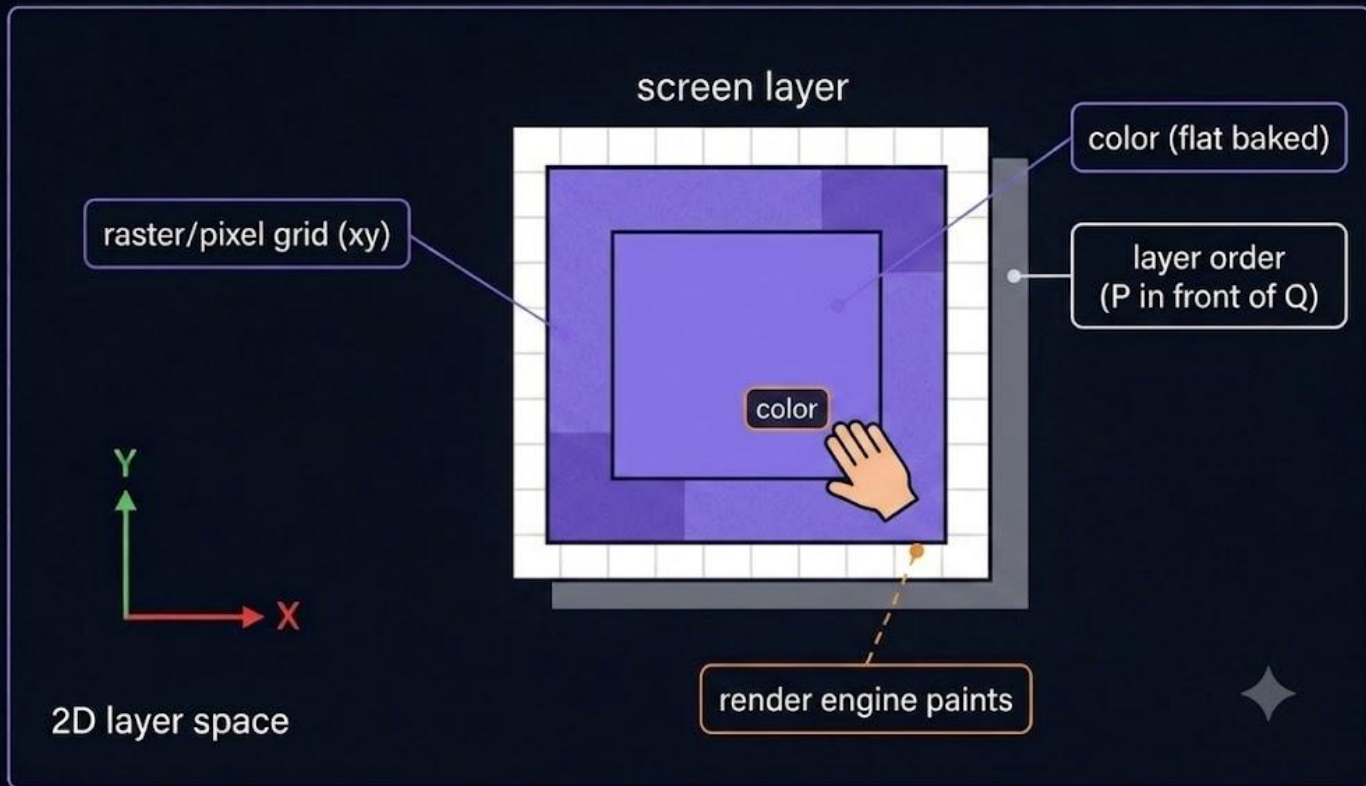
3D Games

- 3D games allow for
 - Immersive experiences
 - Complex gameplay
- Require different file types from 2D games
- Require additional software to run
- 3D games can be built by AI agents ☺



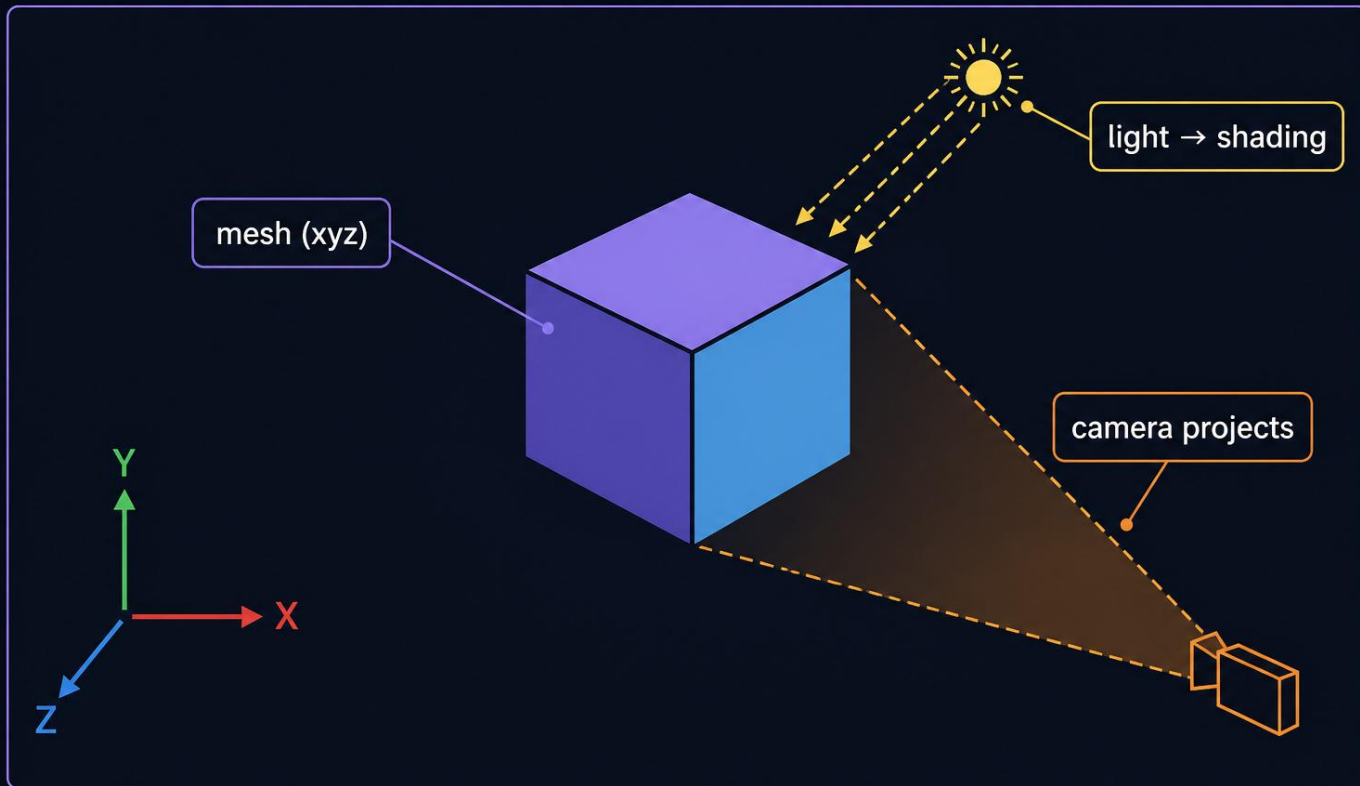
Rendering a 2D Object

2D · plane copied to screen



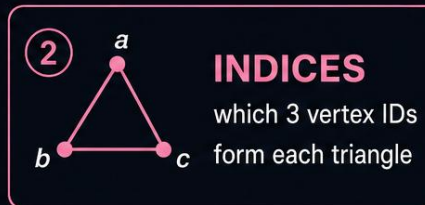
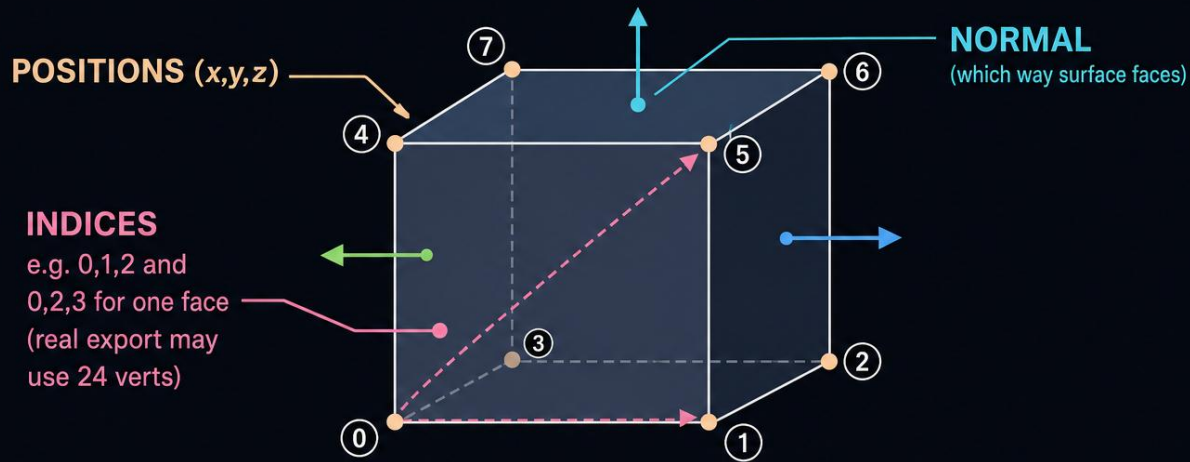
Rendering a 3D Object

3D · volume projected to screen



Elements of a 3D Object

Mesh data: a simple cube



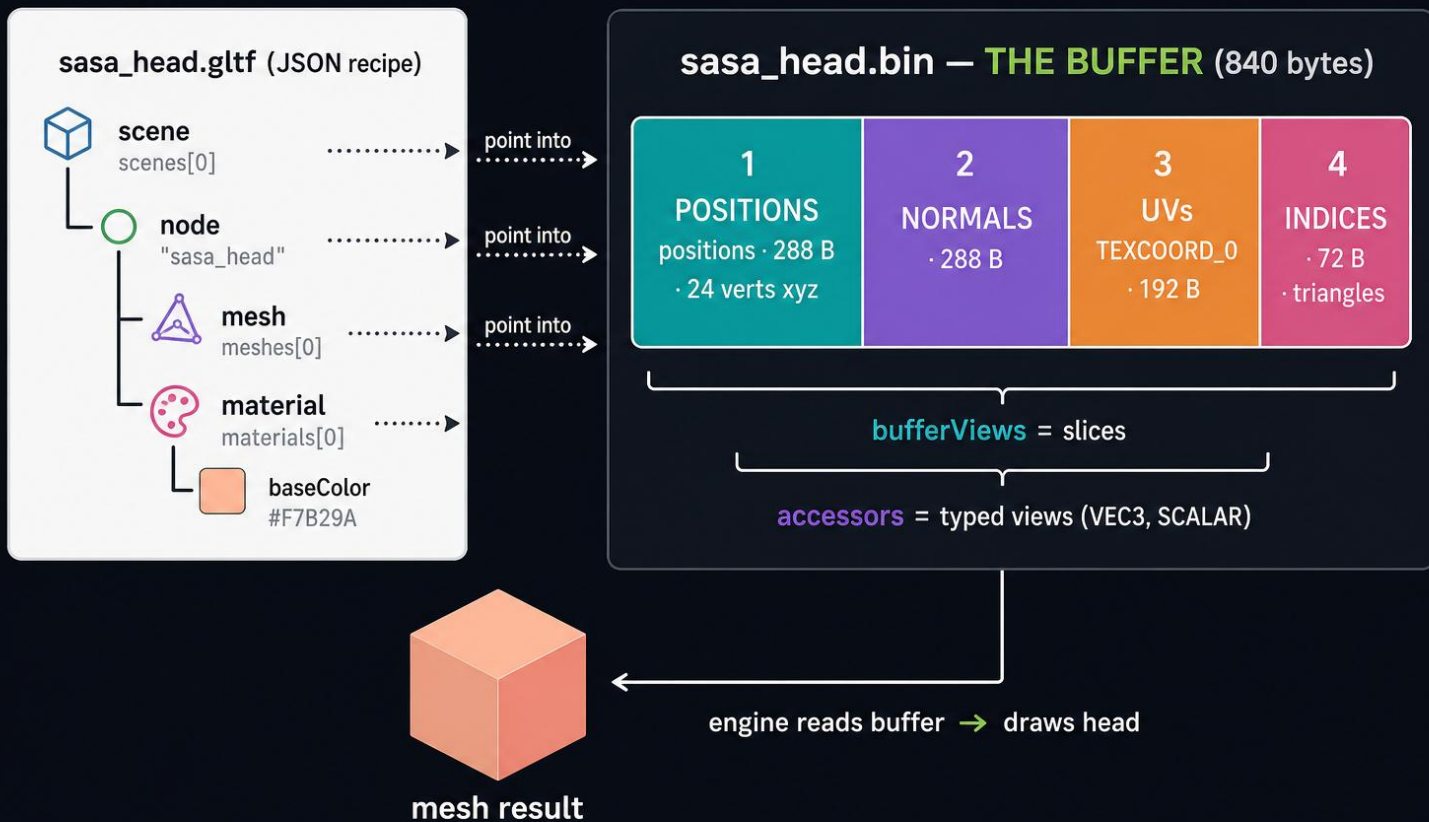
SUMMARY: A mesh = **positions** of vertices + triangles defined by **indices** + outward **normals**.

3D Graphics File Format

- 3D graphics objects have 2 files: **glTF** (graphics library transmission format) and **bin** (binary)
- **glTF** files are human readable recipes for the object:
 - Nodes / hierarchy — parts of the scene (body, head, weapon) and how they parent
 - Meshes — the 3D shapes (vertices + triangles)
 - Materials — base color, roughness, metallic, emission (“how it reacts to light”)
 - Textures (optional) — 2D images mapped onto the mesh
 - Buffers — pointer to raw binary geometry/animation data
- **bin** files tell the graphics engine how to render the object on screen
 - Numbers for mesh data
 - This file holds data that is **buffered** before being rendered

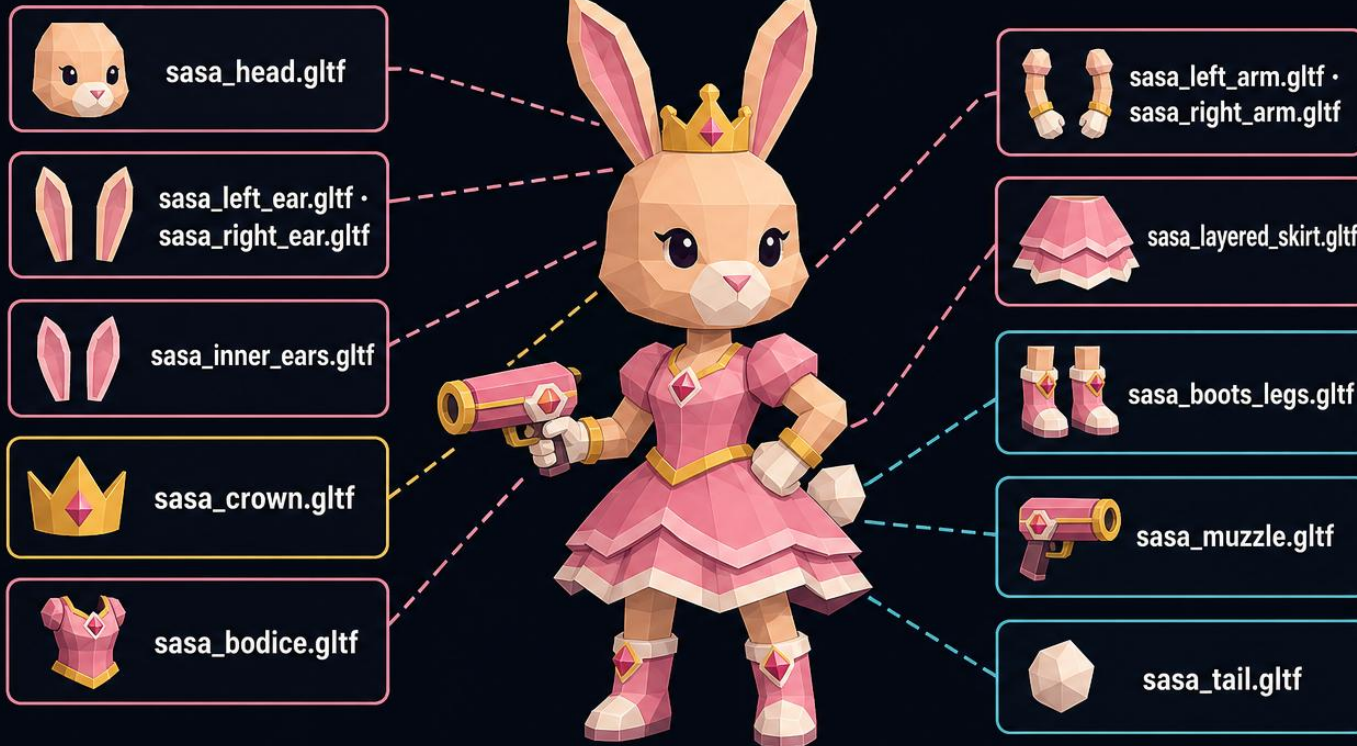
glTF Buffer

glTF buffer: **sasa_head** (terra)



Making Complex Objects

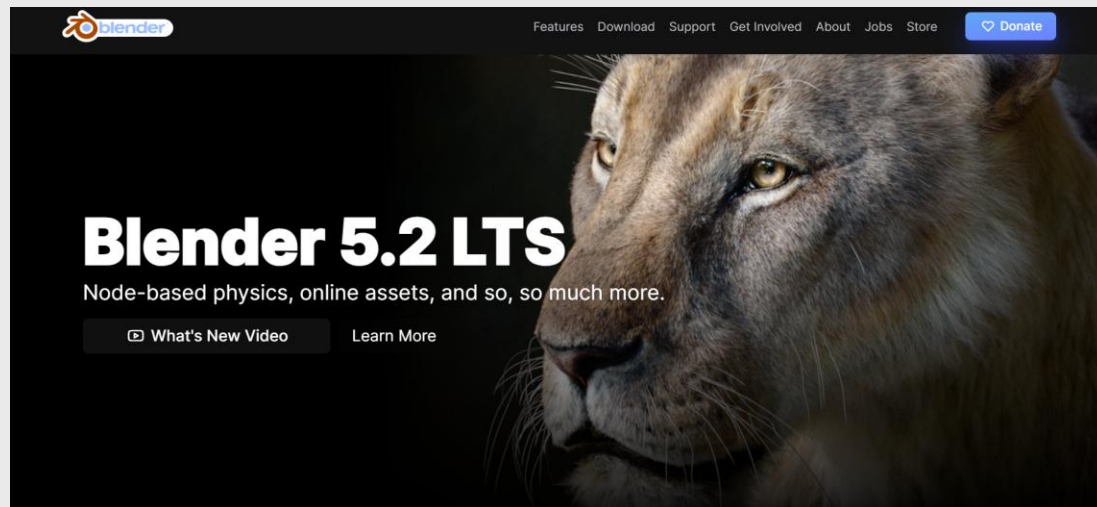
Princess Sasa = many glTF parts (terra)



☆
not one
mega model –
kit of pieces

Blender

- Your agent will not write the glTF and bin files
- Your agent will write python code to generate those files
- Uses a tool called Blender
- Blender has a python package (called bpy), so you can run in with python code
- You need to first install Blender on your machine:
<https://www.blender.org/download/>



Blender Python Scripts

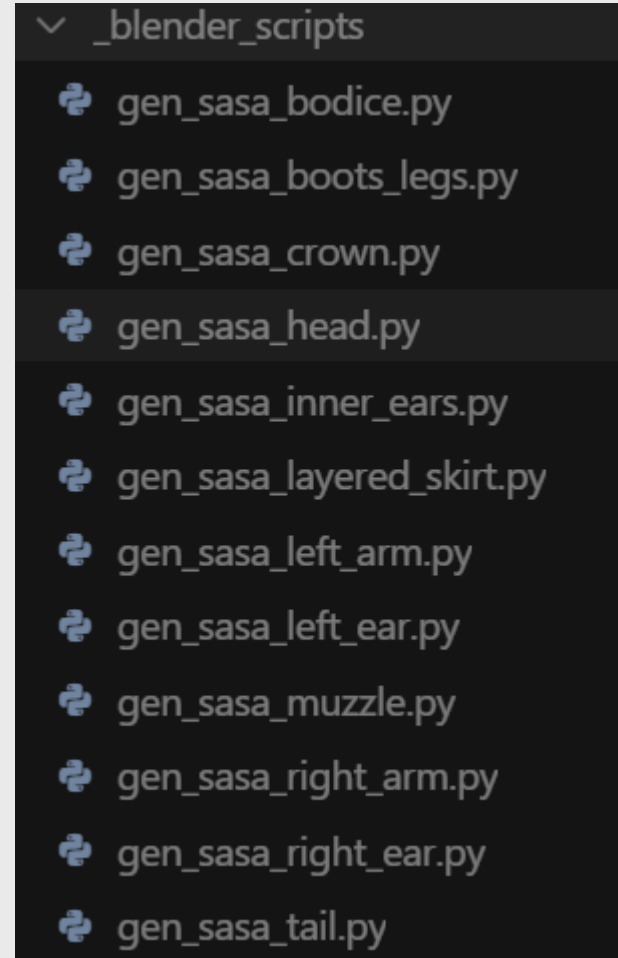
- Example Blender python script

```
import bpy
from pathlib import Path

bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete(use_global=False)
bpy.ops.mesh.primitive_cube_add(size=1.0)
obj = bpy.context.active_object
obj.name = 'sasa_head'
mat = bpy.data.materials.new(name='sasa_head' + "_mat")
mat.use_nodes = True
bsdf = mat.node_tree.nodes.get("Principled BSDF")
if bsdf:
    bsdf.inputs["Base Color"].default_value = (0.9607843137254902, 0.
8352941176470589, 0.7764705882352941, 1.0)
    bsdf.inputs["Roughness"].default_value = 0.45
obj.data.materials.append(mat)
out = 'C:/Users/trz2/Dropbox/ChatGPTs/mgt575_apps/2026SummerSasin/
coding_sessions/games_3d/games/naga_3d_terra/assets/sasa_head.glTF'
Path(out).parent.mkdir(parents=True, exist_ok=True)
bpy.ops.export_scene.glTF(filepath=out, export_format='GLTF_SEPARATE',
use_selection=False)
print("exported", out)
```

Blender Python Scripts

- Your agent will write one python script for each 3D object in your game



3D Game Development Engines

- Once we have the visual and audio assets for the game, we can build the 3D game
- To build the game we need a 3D game development engine (sort of like Phaser was for 2D games)
- Many to pick from
 - **Unity**: The industry stalwart for indie and mobile developers, known for its massive asset store and flexibility (uses C#).
 - **Unreal Engine**: The heavyweight titan for high-end, photorealistic 3D graphics; used heavily in AAA studios (uses C++ and visual Blueprint scripting).
 - **Godot**: Free and lightweight (uses GDScript)

Godot

- Download Godot from its website:
<https://godotengine.org/>
- No installation needed, just unzip the files into a folder, and make sure you agent knows what the folder is

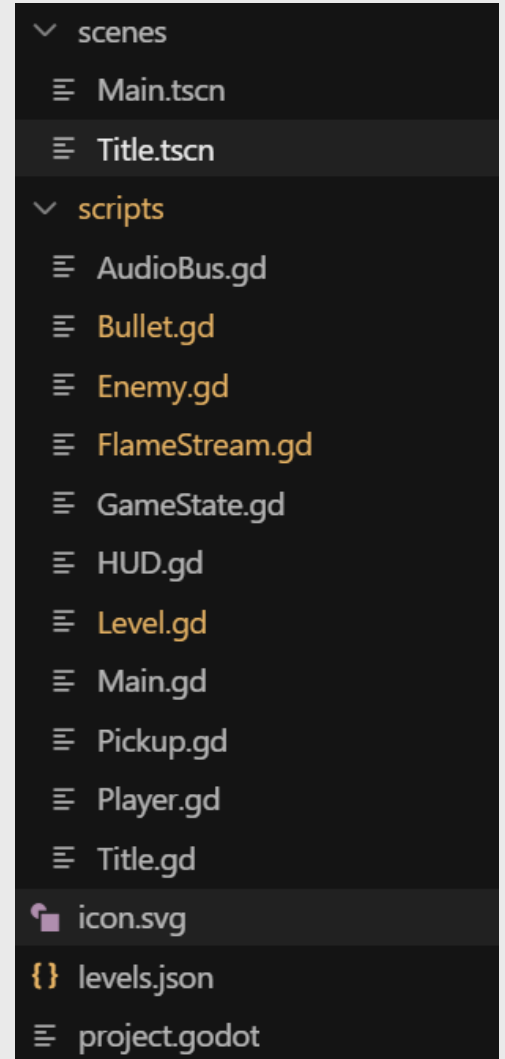


 Godot_v4.7.1-stable_win64

 Godot_v4.7.1-stable_win64_console

Game Assembly

- **Deterministic Python that compiles agent JSON into Godot files**
- **Inputs — Concept, Mechanics, Levels, Art, Story, Audio docs**
- **Writes the game shell — project.godot, scenes, scripts for player, enemies, weapons, levels, HUD**



Hard Quality Control Checks

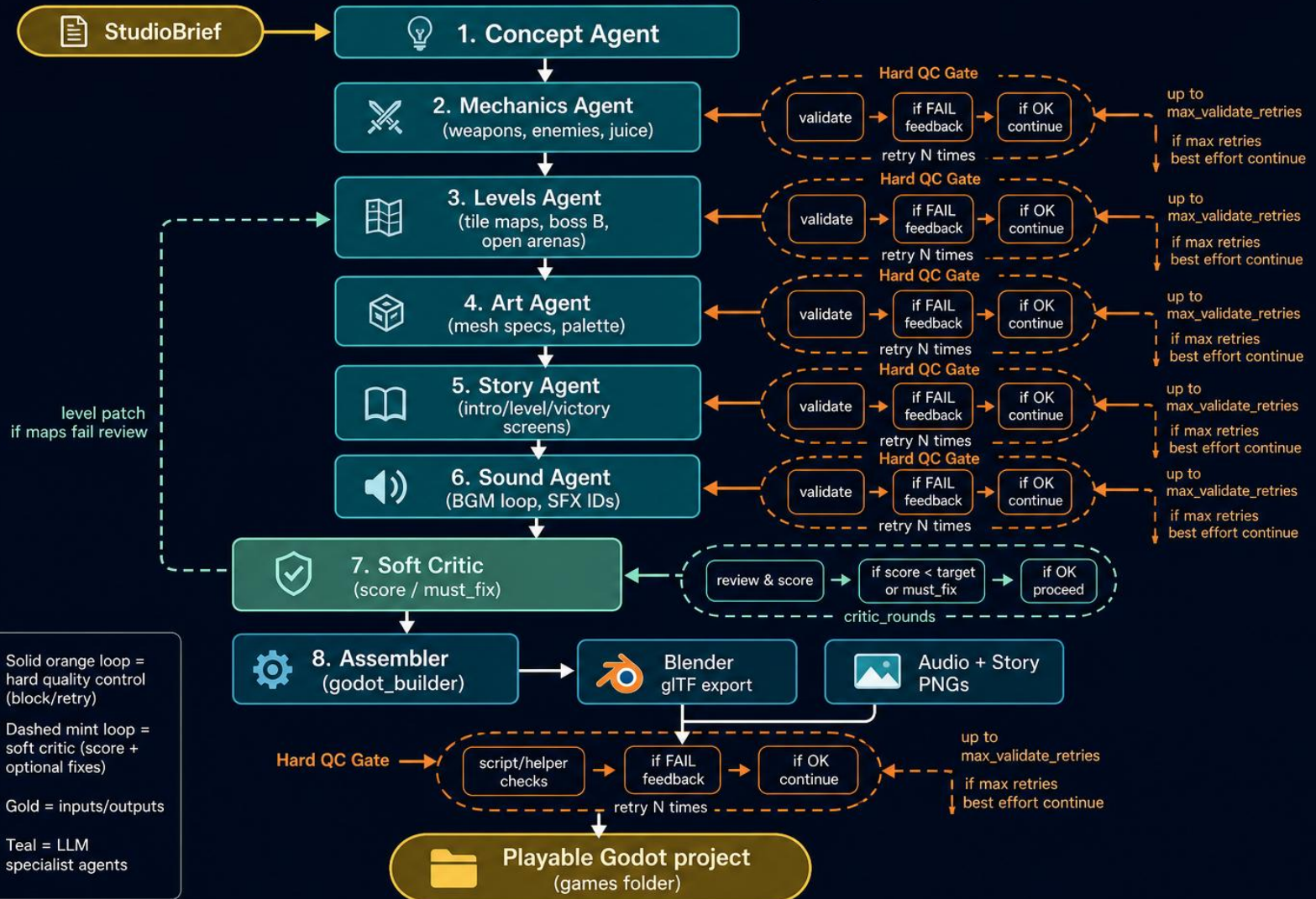
- We can add some hard quality control checks to our agent to catch common errors
- Some common errors (based on personal experience)

Errors (fail the gate)

Check	Why
<code>scripts/Main.gd</code> exists	Main scene must load
Main defines: <code>_load_png</code> , <code>_show_intro</code> , <code>_load_level</code> , <code>_begin_level</code> , <code>_win</code>	Story → arena flow helpers
Main contains <code>level_root.build</code>	Arena actually builds
Main contains <code>play_bgm</code>	Music continues through intros
If Main calls <code>_load_png</code> but doesn't define it	Parse error → grey Main
<code>scripts/Title.gd</code> exists	Entry scene
<code>levels.json</code> exists at project root	Levels after intro

3D Game Agent Flow Diagram

3D Game Studio – Agent Pipeline



Running the Game

- To run the game, you need to know where you put the Godot engine files
- Then you can run a simple terminal command to run the game:
- **"%GODOT_PATH%" –path game_folder**
 - "%GODOT_PATH%" is the location of your Godot engine file
 - game_folder is the folder where the agent put all the game files

Coding Session

